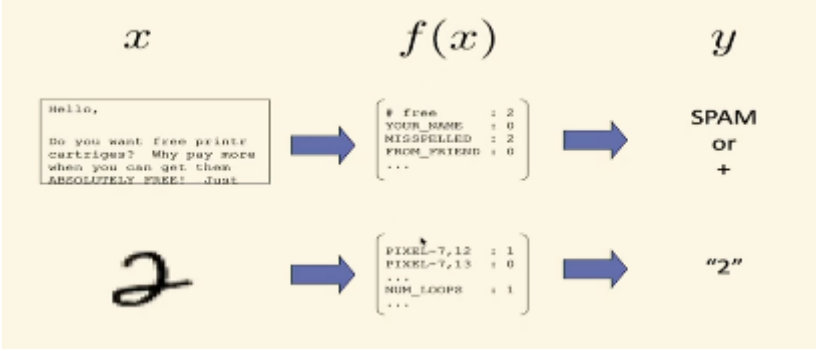
- E' il modello più semplice di rete neurale
  - Cuore: è un algoritmo che, impara iterativamente i pesi che permettono di classificare perfettamente un insieme di esempi iterando su questi e cambiando in maniera deterministica il nostro vettore
  - E' una somma pesata dei propri input
  - Ed a seconda della soglia dei vari input, si da un output
    - Input => funzione => insieme di features => la feature più pesata
    - La feature vector è una lista di feature con dei contatori che serve per decidere l'output
    - 

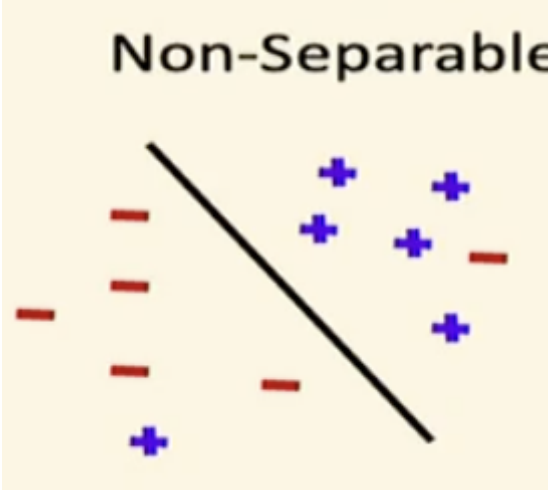
    - La forma più semplice funziona:
      - Input **features value**, ogni feature viene **pesata**, e poi si fa una sommatoria pesata come è l'**attivazione**
      - E poi l'attivazione a seconda di ciò che ha ha 2 valori: positivo (+1), negativo (-1)
      - es:
        - Sigmoid
          - Non ha valori negativi
          - E' costoso
          - Non passa per 0 quando è a 0
          - Ed in più, si può raggiungere gradiente 0, in questo caso il neurone non cambierà più
            - Aka rischia di **uccidere** un neurone dove non imparerà più
        - Tangente iperbolica
          - Passa per 0 ed è negativo
          - E' per il resto stessa di una sigmoid
        - Relu
          - Molto veloce
          - Non è centrato a 0
          - Il gradiente non diventa mai troppo piccolo

        - Però anche qui, è possibile che si arrivi gradiente 0 ed il neurone morirà
    - Leaky relu
        - E' un relu che però non può morire
        - Però qui siccome cresce troppo, il gradiente potrebbe crescere veramente tanto ed andare in overflow
    - ELU
        - Se x è > 0 allora x, senò è una esponenziale
        - Ha tutti i benefici della Relu, senza esplodere nè nulla, però ha un esponenziale che è costosa
          Tldr:
    - Usare relu per classificazioni, senò usare leaky relu se vuoi avere un piccolo margine
- Esempio di features: se abbiamo un immagine, possiamo pensare di semplificarla
    - Togliere i colori
    - approfondire il contrasto
    - Sottolineare le diverse parti del corpo
    - Oppure potremmo persino dare x^2, e nota bene:
        - Questa è una elaborazione che viene fatta prima di darlo alla rete neurale.
        - Potremmo anche dire, $< x_1, x_2, x_1 * x_2, x^3, x^4 >$
          Facendo così permettiamo agli input di non essere per forza lineari
- Il percettore è un esempio di classificatore lineare
    - E' un prodotto scalare, ed a seconda dell'angolazione del prodotto scalare si sa se è positivo o negativo
      Se vicino 90 gradi positivo, senò 180 gradi
    - Si aggiunge a volte una feature bias/offset
        - E serve per evitare che le linee passano da 0 0
        - E' sempre 1
- Come si allena un percettore:
    - Si prendono i pesi attuali, si moltiplica per le features
        - Se ci viene data una classificazione corretta, non cambiamo i pesi
        - Se viene sbagliata la classificazione, cambiamo i pesi
            - Per farlo sottraiamo il vettore delle features da quelle dei pesi

$$= \begin{cases} +1 & \text{se } f(x) \geq 0 \\ -1 & \text{se } f(x) < 0 \end{cases} = +y^* \ f(x)$$

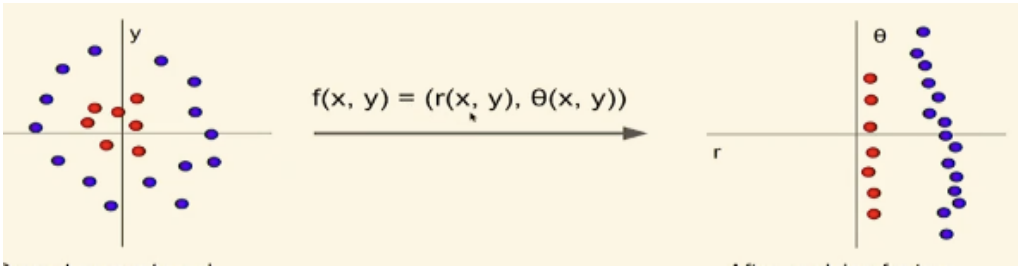
- Problemi:

- Il dataset non è separabile

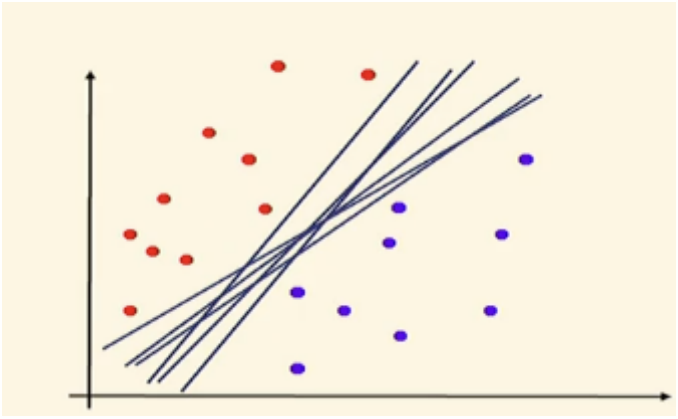


Così l'algoritmo continuerà all'infinito e non potrà mai finire

  - Soluzione: aggiungere nuove features, aka aumentare la dimensionalità delle features
  - Soluzione: usare una funzione che mappa le features attraverso una funzione di trasformazione delle features
    - Es: togliere colori da una immagine



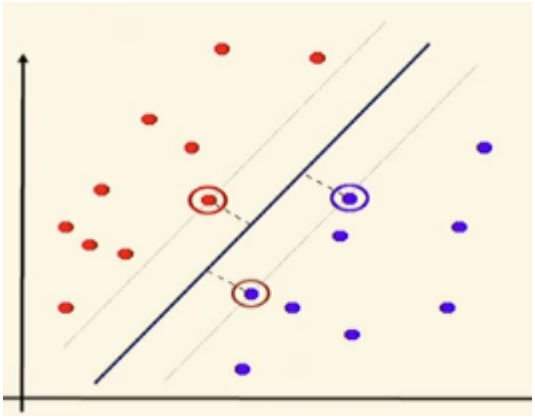
- Ci sono tante linee che sono accettabili, quale si accetta?



La risposta ce la da la support vector machine (MIRA e SVM) (capita in esame)

  - Ci trova un separatore lineare che ha margine massimo

- E' una ottimizzazione



Ci da un insieme di pesi con margine massimo